

2.4 Multivariate Examples

In the previous notebook, we used gradient methods to find minimum of function of one variable. In this notebook, we will use the previous methods to find location of the minimum of functions of two variables.

```
In [1]: using CairoMakie
using LinearAlgebra
```

Declare structs to store the store the intermediate solutions

```
In [2]: struct Point2D
        x1::Float64
        x2::Float64
        fvalue::Float64
end
```

Example 2.4.1

- Let's now try to find the minimum of a two-dimensional function $f(x) = x_1^2 + x_2^2$.
- As was seen in the previous examples, to use gradient methods to find the minimum of a function, you need the gradients. Since this is a two dimensional function, its gradient is a two dimensional vector,
$$\nabla f = \vec{g} = \begin{Bmatrix} \partial f / \partial x_1 \\ \partial f / \partial x_2 \end{Bmatrix} = \begin{Bmatrix} 2x_1 \\ 2x_2 \end{Bmatrix}$$
- This function has a local minimum at $(x_1, x_2) = (0.0, 0.0)$
- Since this is a two dimensional function, we can visualise using a a contour plot

```
In [3]: x1range=-5:0.02:5;
x2range=-5:0.02:5;
bowl(x)=x[1]^2+x[2]^2

function Gradient_bowl(x)
    d1=2*x[1]
    d2=2*x[2]
    return [d1,d2]
end

funcplot = [bowl([x1,x2]) for x1 in x1range, x2 in x2range]

x = [(x1i, x2i) for x1i in x1range, x2i in x2range]
fig = Figure(size=(800, 600))
ax = Axis(fig[1, 1], xlabel = L"x_1", ylabel = L"x_2", title = "Bowl funct
levels = range(0.01, 50; length=6)
```

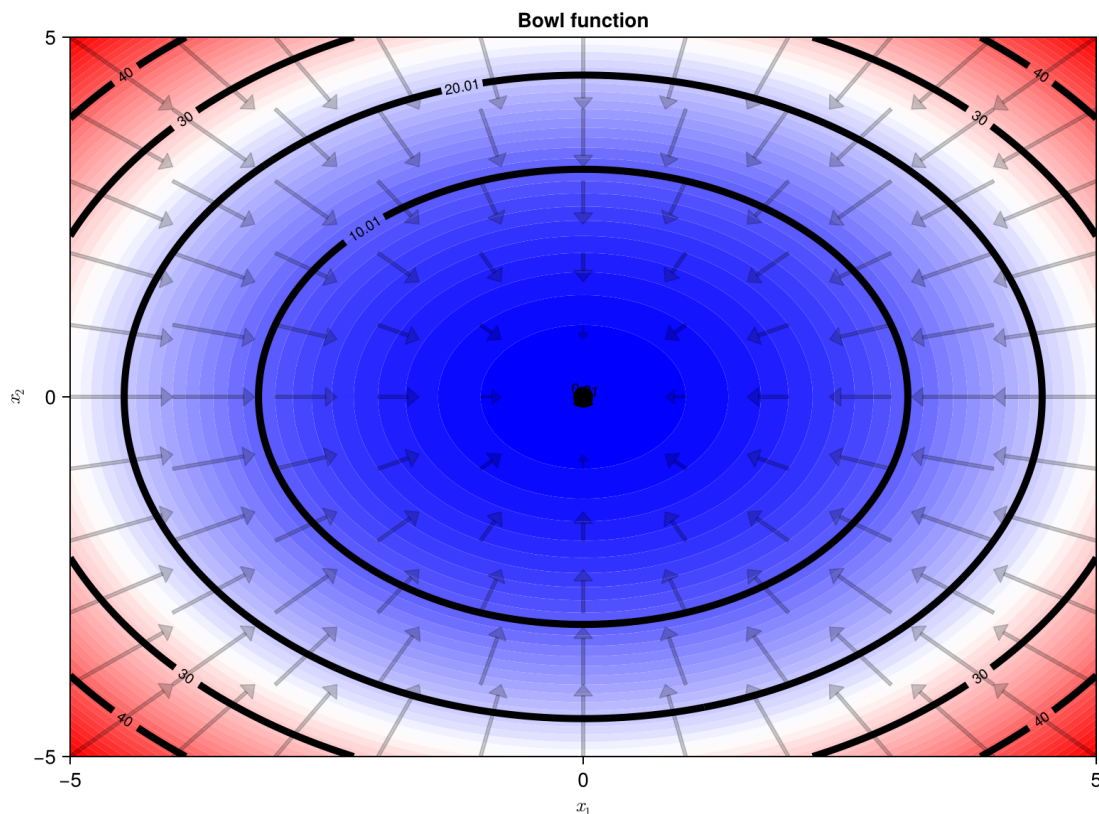
```

contourf!(ax,x1range,x2range,funcplot; levels=50, colormap=:bwr)
contour!(ax,x1range,x2range,funcplot; labels=true, levels, color=:black,
scatter!(ax,[0],[0],color=:black,markersize=20)

x1range_arrow=-5:1:5;
x2range_arrow=-5:1:5;
arrowplot = [Gradient_bowl([x1,x2]) for x1 in x1range_arrow, x2 in x2rang
arrows2d!(ax,x1range_arrow,x2range_arrow,-getindex.(arrowplot,1),-getinde

# Save as PNG
save("../figures/my_plot.png", fig)
display(fig)

```



CairoMakie.Screen{IMAGE}

- The arrows in the plot above shows the direction of the negative of the gradient of $f(x)$, $-\nabla f(x)$.
- Note that all the arrows point to the direction of the minimum of the function $f(x)$. So if we just "follow" the arrows, we should move towards the local minimum.
- This is the main idea behind the standard gradient descent method, to always move in the the opposite direction of the gradient of the function you are trying to optimize.

Example 2.4.1 with Standard Gradient Descent

Try standard gradient descent method with update equations given by

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} + \alpha \vec{g}^{(k)}$$

Recall that α is a constant and all variables with an arrow on top are vectors in this notebook.

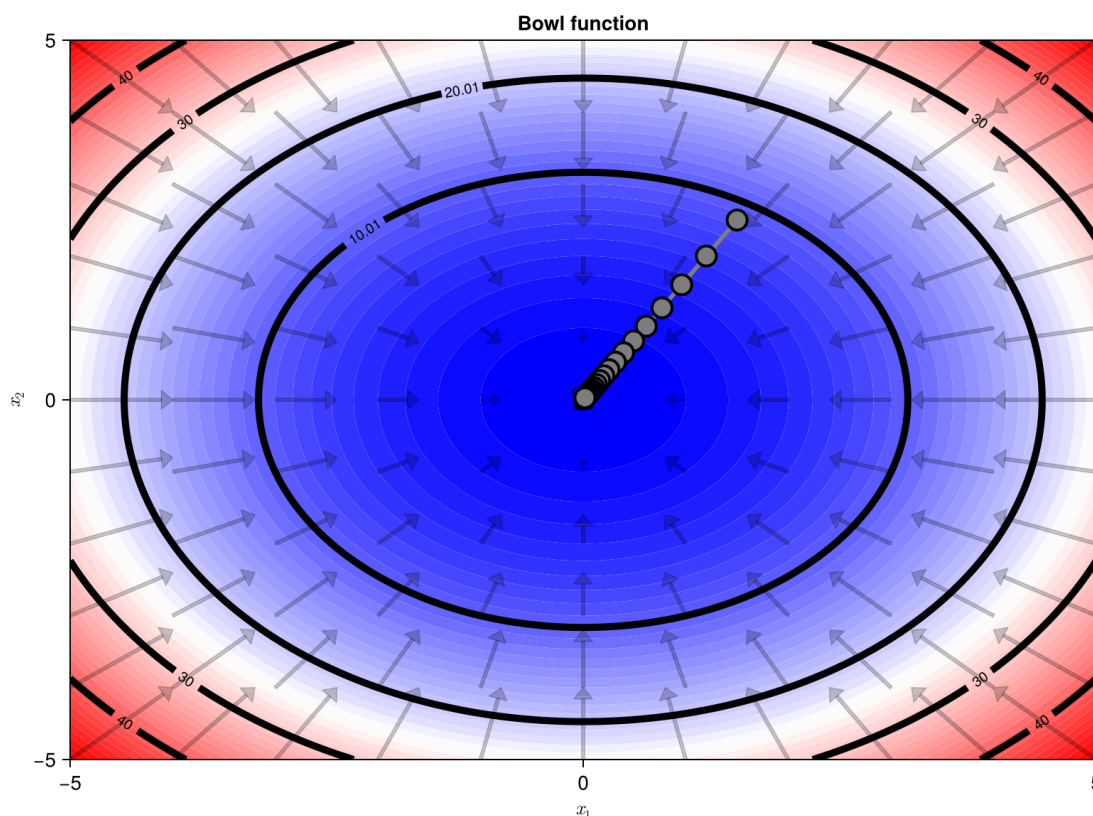
Written explicitly, the gradient descent method for a function of two variables $f(x_1, x_2)$ is shown below

$$\begin{Bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \end{Bmatrix} = \begin{Bmatrix} x_1^{(k)} \\ x_2^{(k)} \end{Bmatrix} + \alpha \begin{Bmatrix} \frac{\partial f}{\partial x_1}(x_1^{(k)}, x_2^{(k)}) \\ \frac{\partial f}{\partial x_2}(x_1^{(k)}, x_2^{(k)}) \end{Bmatrix} \quad (1)$$

The code below uses standard gradient descent to find the local minimum.

```
In [4]: x0=[1.5,2.5] #initial guess x0. Since this is a two-dimensional function
alpha=0.1 #learning rate
data=[Point2D(x0...,bowl(x0))]
x=x0
for i in 1 : 20
    x-=alpha*Gradient_bowl(x)
    push!(data,Point2D(x...,bowl(x)))
end
lines!(ax,getproperty.(data, :x1),getproperty.(data, :x2);linewidth=3,col
scatter!(ax,x0[1],x0[2],color=:blue,markersize=20)
scatter!(ax,getproperty.(data, :x1),getproperty.(data, :x2);color=:grey,m

# Save as PNG
save("../figures/my_plot_02.png", fig)
display(fig)
```



CairoMakie.Screen{IMAGE}

- The gradient descent method (starting from the blue dot), follows the arrows (which is the negative of the gradient) and goes towards the local minimum (black dot)
- This is a very easy example and the standard gradient descent works very well. We will now look at more complicated examples below.

Example 2.4.2

Try "Simple" quadratic function from "An Introduction to the Conjugate Gradient Method Without the Agonizing Pain" Edition 1 1 4 Jonathan Richard Shewchuk August 4, 1994

$$f(x_1, x_2) = \frac{3}{2}x_1^2 + 2x_1x_2 + 3x_2^2 - 2x_1 + 8x_2$$

Exercise Ex01: Compute $\partial f / \partial x_1$ and $\partial f / \partial x_2$ and set them to zero to show that the minimum of this function occurs at $(x_1, x_2) = (2, -2)$.

```
In [5]: quad(x)=(3.0/2.0)*x[1]^2+2*x[1]*x[2]+3*x[2]^2-2*x[1]+8*x[2] # Define the
function Gradient_quad(x)
    d1=3*x[1]+2*x[2]-2 #df/dx1
    d2=6*x[2]+2*x[1]+8 #df/dx2
    return [d1,d2]
end

function Hessian_quad(x)
    return [3 2;2 6] # This is the Hessian of the function which we will
end

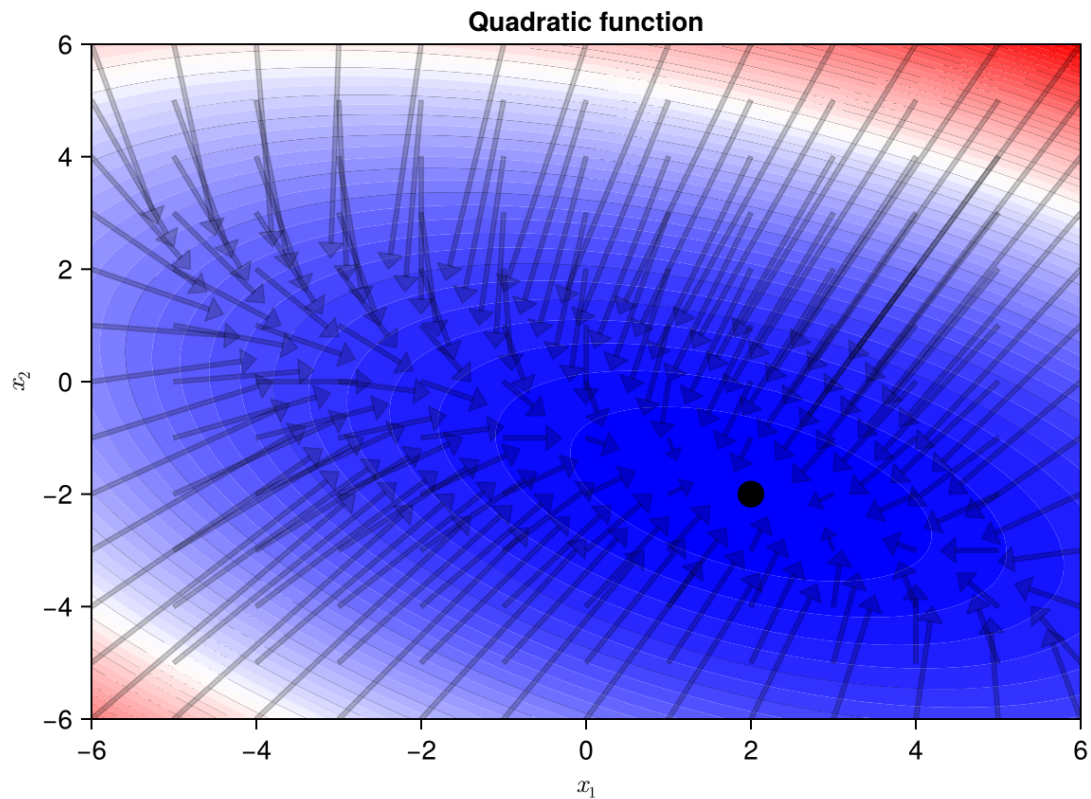
x1range=-6:0.02:6;
x2range=-6:0.02:6;
funcplot = [quad([x1,x2]) for x1 in x1range, x2 in x2range]

fig = Figure()
ax = Axis(fig[1, 1], xlabel = L"x_1", ylabel = L"x_2",title = "Quadratic
contour!(ax,x1range,x2range,funcplot; labels=true, levels=20,color=:black
contourf!(ax,x1range,x2range,funcplot; levels=50,colormap=:bwr)

x1range_arrow=-6:1:6;
x2range_arrow=-6:1:6;
arrowplot = [Gradient_quad([x1,x2]) for x1 in x1range_arrow, x2 in x2rang
arrows2d!(ax,x1range_arrow,x2range_arrow,-getindex.(arrowplot,1),-getinde

scatter!(ax, [2], [-2],color=:black,markersize=20)

# Save as PNG
save("../figures/my_plot_03.png", fig)
display(fig)
```



CairoMakie.Screen{IMAGE}

Example 2.4.2 with Standard Gradient Descent

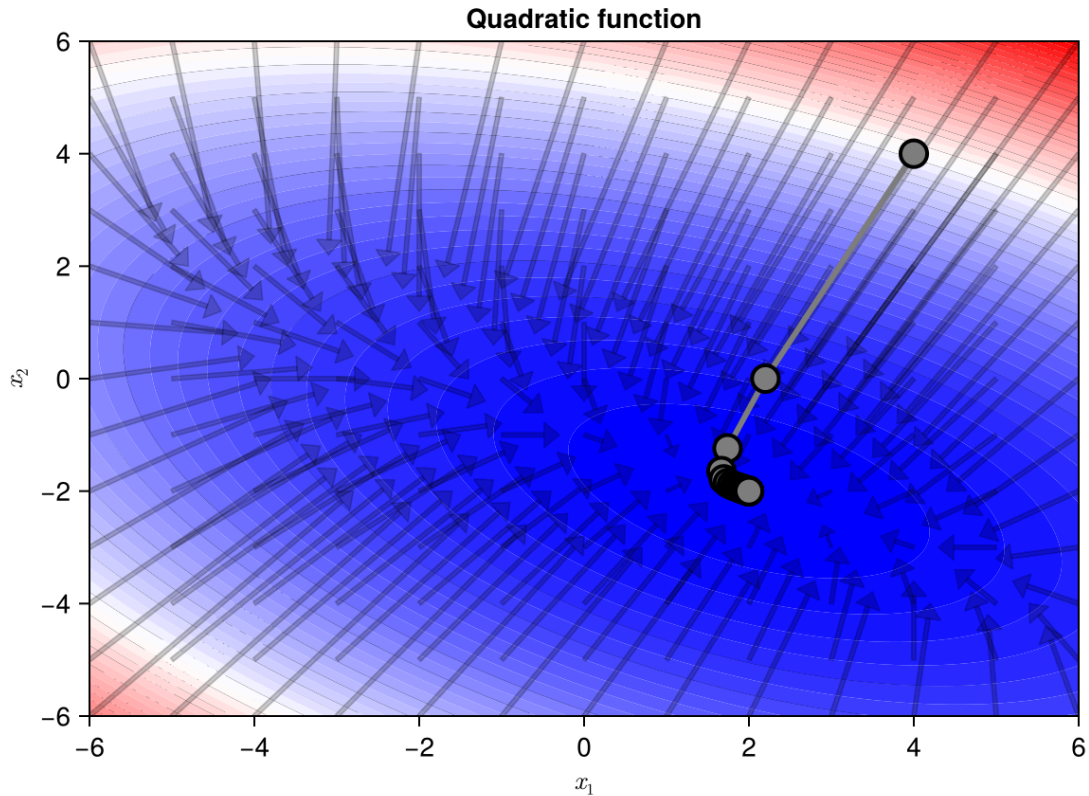
The code below uses the standard gradient descent method to find the local minimum of the quadratic function defined above.

```
In [6]: x0=[4.0,4.0] #initial guess of xm
alpha=0.1 #learning rate
data=[Point2D(x0...,quad(x0))]

x=x0

for i in 1 : 50
    x-=alpha*Gradient_quad(x)
    push!(data,Point2D(x...,quad(x)))
end

lines!(ax,getproperty.(data, :x1),getproperty.(data, :x2);linewidth=3,col
scatter!(ax,getproperty.(data, :x1),getproperty.(data, :x2);color=:grey,m
#scatter!(ax,x0[1],x0[2],color=:blue,markersize=20)
fig
```



Example 2.4.2 with Momentum

The update equations for the momentum method for a function of two variables are

$$\vec{v}^{(k+1)} = \beta \vec{v}^{(k)} - \alpha \vec{g}^{(k)} \quad (2)$$

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} + \vec{v}^{(k+1)} \quad (3)$$

where $\vec{g}^{(k)} = \nabla f(x^{(k)})$, α is the learning rate and β is the momentum decay.

Sometimes writing things in vector form hides the details of what is going on. It is useful to write it out explicitly and in two dimensions, the update equations for the momentum equations are

$$\begin{Bmatrix} v_1^{(k+1)} \\ v_2^{(k+1)} \end{Bmatrix} = \beta \begin{Bmatrix} v_1^{(k)} \\ v_2^{(k)} \end{Bmatrix} - \alpha \begin{Bmatrix} \frac{\partial f}{\partial x_1}(x_1^{(k)}, x_2^{(k)}) \\ \frac{\partial f}{\partial x_2}(x_1^{(k)}, x_2^{(k)}) \end{Bmatrix} \quad (4)$$

$$\begin{Bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \end{Bmatrix} = \begin{Bmatrix} x_1^{(k)} \\ x_2^{(k)} \end{Bmatrix} + \begin{Bmatrix} v_1^{(k+1)} \\ v_2^{(k+1)} \end{Bmatrix} \quad (5)$$

The Julia code to implement the momentum equations is given below

```
In [7]: x0=[4.0,4.0] #initial guess of x0
alpha=0.1 #learning rate
beta=0.4 #momentum
v=[0.0,0.0]
data_m=[Point2D(x0...,quad(x0))]
x=x0
```

```

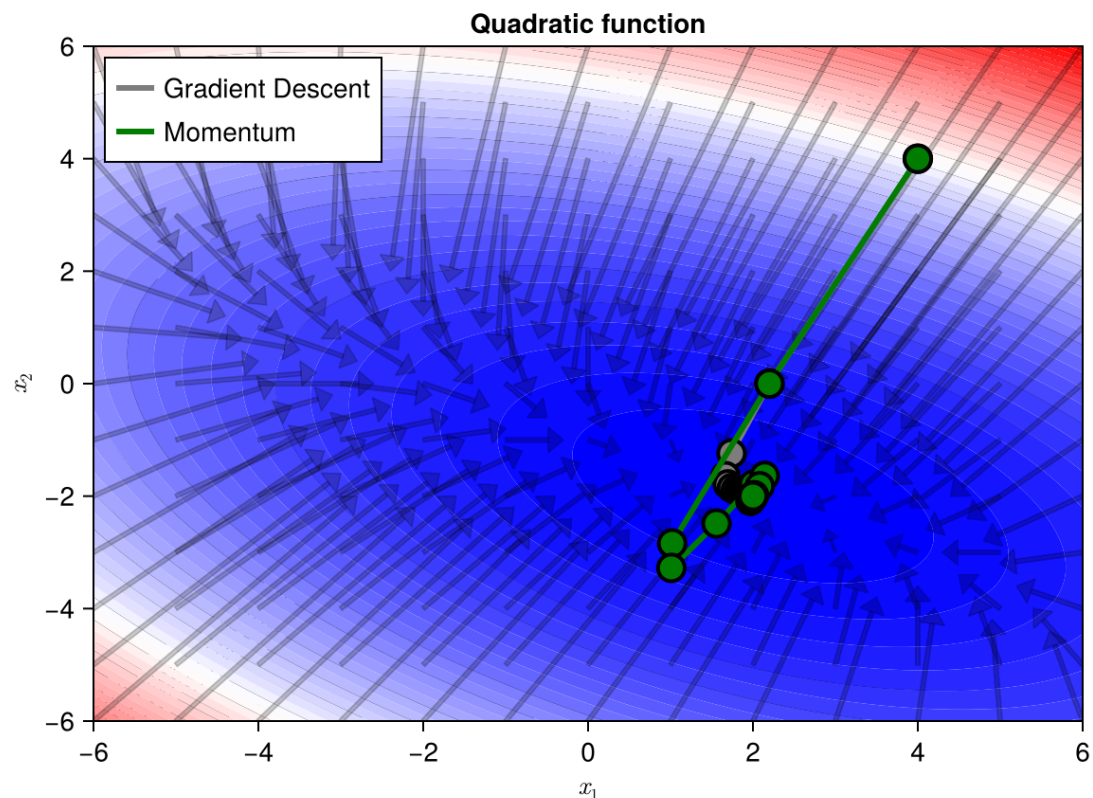
for i in 1 : 20
    v=beta*v.-alpha*Gradient_quad(x)
    x+=v
    push!(data_m,Point2D(x...,quad(x)))
end

lines!(ax,getproperty.(data_m, :x1),getproperty.(data_m, :x2),linewidth=3
scatter!(ax,getproperty.(data_m, :x1),getproperty.(data_m, :x2);color=:gr

axislegend(ax;position = :lt)

# Save as PNG
save("../figures/my_plot_04.png", fig)
display(fig)

```



CairoMakie.Screen{IMAGE}

Example 2.4.2 with Nesterov Momentum.

Let's use Nesterov momentum. The update equations for the Nesterov momentum method for a function of two variables are

$$\vec{v}^{(k+1)} = \beta \vec{v}^{(k)} - \alpha \vec{g}^{(k)} \quad (6)$$

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} + \vec{v}^{(k+1)} \quad (7)$$

where $\vec{g}^{(k)} = \nabla f(x^{(k)} + \beta v^{(k)})$

For two variables, the Nesterov Momentum update equations can be written as

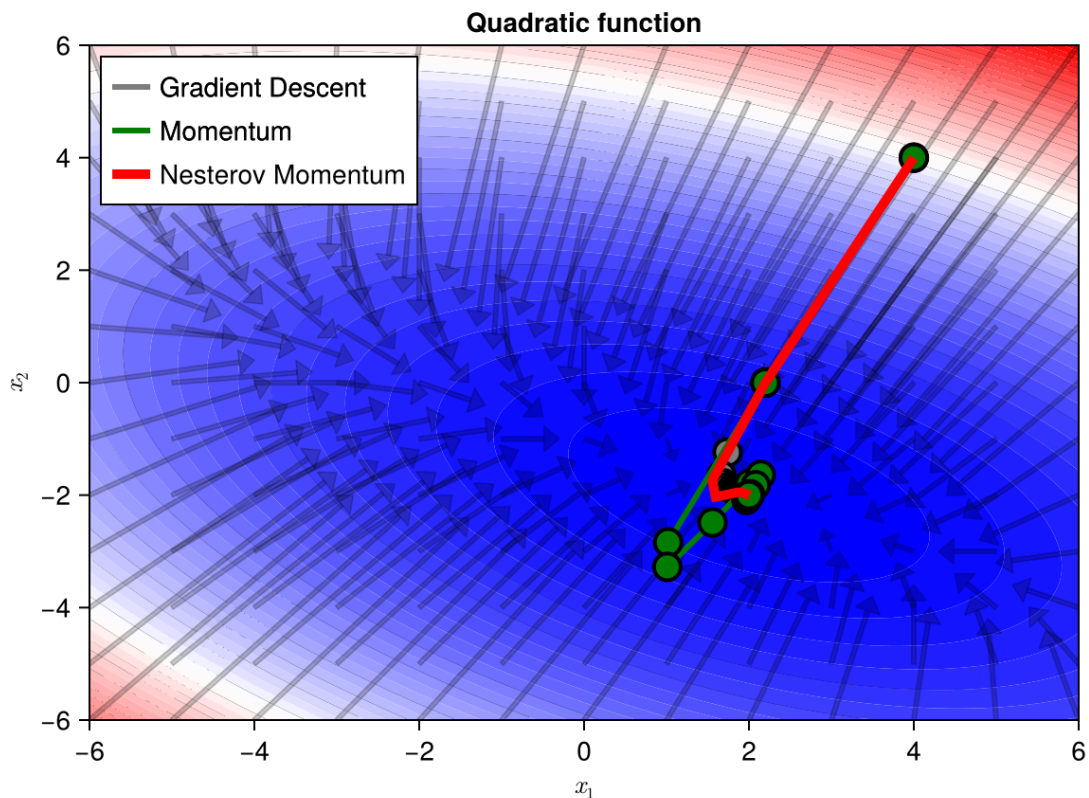
$$\begin{Bmatrix} v_1^{(k+1)} \\ v_2^{(k+1)} \end{Bmatrix} = \beta \begin{Bmatrix} v_1^{(k)} \\ v_2^{(k)} \end{Bmatrix} - \alpha \begin{Bmatrix} \frac{\partial f}{\partial x_1}(x_1^{(k)} + \beta v_1^{(k)}, x_2^{(k)} + \beta v_2^{(k)}) \\ \frac{\partial f}{\partial x_2}(x_1^{(k)} + \beta v_1^{(k)}, x_2^{(k)} + \beta v_2^{(k)}) \end{Bmatrix} \quad (8)$$

$$\begin{Bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \end{Bmatrix} = \begin{Bmatrix} x_1^{(k)} \\ x_2^{(k)} \end{Bmatrix} + \alpha \begin{Bmatrix} v_1^{(k+1)} \\ v_2^{(k+1)} \end{Bmatrix} \quad (9)$$

The Julia code for the Nesterov momentum method is shown below.

```
In [8]: x0=[4.0,4.0] #initial guess of xm
alpha=0.1 #learning rate
beta=0.4
v=[0.0,0.0]
x=x0
data_nm=[Point2D(x0...,quad(x0))]
for i in 1 : 20
    v =beta*v-alpha*Gradient_quad(x+beta*v)
    x+=v
    push!(data_nm,Point2D(x...,quad(x)))
end

lines!(ax,getproperty.(data_nm, :x1),getproperty.(data_nm, :x2),linewidth
axislegend(ax;position = :lt)
fig
```



- As expected the Nesterov momentum method does not "bounce around" as much as the Momentum method.

Example 2.4.2 with Newton Method

The update equations for the Newton method for a multivariable function is not as straightforward as the methods shown above. However, we can derive it for a function of two variables and then extending it to multi dimensions should be straightforward. Consider the Taylor series expansion (truncated at 2nd order) for a function of two variables

$$\begin{aligned} f(x_1, x_2) \approx & f(x_{01}, x_{02}) + \frac{\partial f}{\partial x_1}(x_{01}, x_{02})(x_1 - x_{01}) + \frac{\partial f}{\partial x_2}(x_{01}, x_{02})(x_2 - x_{02}) \\ & + \frac{\partial^2 f}{\partial x_2 \partial x_1}(x_{01}, x_{02})(x_1 - x_{01})(x_2 - x_{02}) \\ & + \frac{1}{2!} \frac{\partial^2 f}{\partial x_1^2}(x_{01}, x_{02})(x_1 - x_{01})^2 + \frac{1}{2!} \frac{\partial^2 f}{\partial x_2^2}(x_{01}, x_{02})(x_2 - x_{02})^2 \end{aligned}$$

If we want to find the minimum of the function $f(x_1, x_2)$, we need to find (x_1, x_2) where

$$\frac{\partial f}{\partial x_1} = 0 \text{ and } \frac{\partial f}{\partial x_2} = 0.$$

Differentiate the 2nd order Taylor series expansion and set to zero gives

$$\frac{\partial f}{\partial x_1}(x_1, x_2) = \frac{\partial f}{\partial x_1}(x_{01}, x_{02}) \quad (13)$$

$$+ \frac{\partial^2 f}{\partial f \partial x_1}(x_{01}, x_{02})(x_2 - x_{02}) \quad (14)$$

$$+ \frac{\partial^2 f}{\partial x_1^2}(x_{01}, x_{02})(x_1 - x_{01}) \quad (15)$$

$$= 0 \quad (16)$$

$$\frac{\partial f}{\partial x_2}(x_1, x_2) = \frac{\partial f}{\partial x_2}(x_{01}, x_{02}) \quad (17)$$

$$+ \frac{\partial^2 f}{\partial x_2 \partial x_1}(x_{01}, x_{02})(x_1 - x_{01}) \quad (18)$$

$$+ \frac{\partial^2 f}{\partial x_2^2}(x_{01}, x_{02})(x_2 - x_{02}) \quad (19)$$

$$= 0 \quad (20)$$

Solving for x_1 and x_2 gives the update equations for the Newton method

$$\begin{Bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \end{Bmatrix} = \begin{Bmatrix} x_1^{(k)} \\ x_2^{(k)} \end{Bmatrix} - \begin{bmatrix} H_{11} & H_{12} \\ H_{21} & H_{22} \end{bmatrix}^{-1} \begin{Bmatrix} \frac{\partial f}{\partial x_1}(x_1^{(k)}, x_2^{(k)}) \\ \frac{\partial f}{\partial x_2}(x_1^{(k)}, x_2^{(k)}) \end{Bmatrix} \quad (21)$$

In general for a function of N variables, Newton method can be written as

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} - \left(\mathbf{H}^{(k)} \right)^{-1} \vec{g}^{(k)}$$

where $\mathbf{H}$ is the $N \times N$ Hessian matrix

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

and $\vec{x}$ is an $N \times 1$ vector.

The implementation of the Newton method in Julia is given below

```
In [9]: x0=[4.0,4.0]#initial guess of xm

data_Newton=[Point2D(x0...,quad(x0))]
x=x0
for i in 1 : 20
    x-=Hessian_quad(x)\Gradient_quad(x)
    push!(data_Newton,Point2D(x...,quad(x)))
end

data_Newton
```

21-element Vector{Point2D}:

```
Point2D(4.0, 4.0, 128.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
Point2D(2.0, -2.0, -10.0)
```

As you can see from the example above, the Newton method converges very quickly for this method. After only 1 iteration for this example.

Example 2.4.3

Let's now try the Himmelblau Function

$$f(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$$

This function has one local maxima at $(x_1, x_2) = (-0.270845..., -0.923039...)$ and 4 local minima at $(x_1, x_2) = (3.0, 2.0), (-2.805118..., 3.13132...), (-3.779310..., -3.283136), (3.584...)$.

The derivatives are

$$\frac{\partial f}{\partial x_1}(x_1, x_2) = 4x_1(x_1^2 + x_2 - 11) + 2(x_1 + x_2^2 - 7)$$

$$\frac{\partial f}{\partial x_2}(x_1, x_2) = 2(x_1^2 + x_2 - 11) + 4x_2(x_1 + x_2^2 - 7)$$

and the components of the Hessian are

$$\frac{\partial^2 f}{\partial x_1^2}(x_1, x_2) = 12x_1^2 + 4x_2 - 42$$

$$\frac{\partial^2 f}{\partial x_1 \partial x_2}(x_1, x_2) = 4x_1 + 4x_2$$

$$\frac{\partial^2 f}{\partial x_2 \partial x_1}(x_1, x_2) = 4x_1 + 4x_2$$

$$\frac{\partial^2 f}{\partial x_2^2}(x_1, x_2) = 4x_1 + 12x_2^2 - 26$$

Define the Himmelblau function, its gradients and Hessian and plotting it to see how it looks like.

```
In [10]: himmelblau(x)=(x[1]^2+x[2]-11)^2+(x[1]+x[2]^2-7)^2

function Gradient_himmelblau(x)
    d1=4*x[1]*(x[1]^2+x[2]-11)+2*(x[1]+x[2]^2-7)
    d2=2*(x[1]^2+x[2]-11)+4*x[2]*(x[1]+x[2]^2-7)
    return [d1,d2]
end

function Hessian_himmelblau(x)
    return [12*x[1]^2+4*x[2]-42 4*x[1]+4*x[2];4*x[1]+4*x[2] 12*x[2]^2+4*x[1]-26]
end

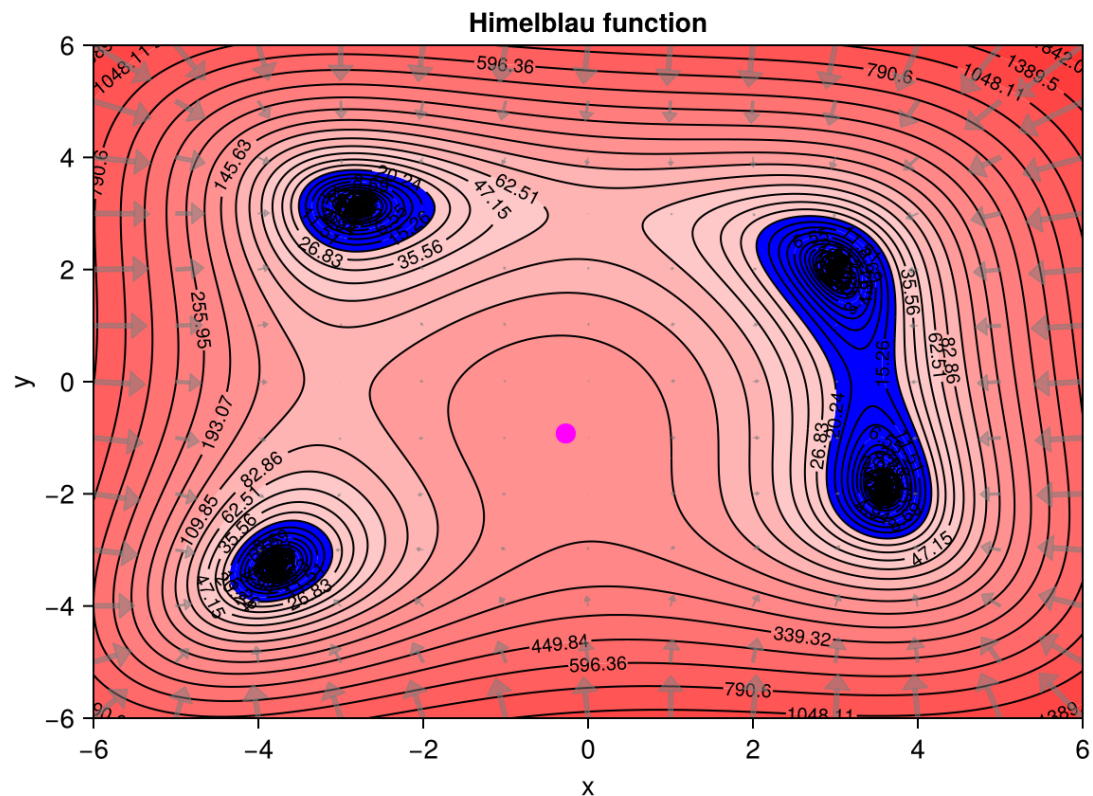
x1range=-6:0.02:6;
x2range=-6:0.02:6;

funcplot = [himmelblau([x1,x2]) for x1 in x1range, x2 in x2range]

fig = Figure()
ax = Axis(fig[1, 1], xlabel = "x", ylabel = "y", title = "Himmelblau function")
levels = 10.0.^range(-2, 4; length=50)
contourf!(ax,x1range,x2range,funcplot; levels,colormap=:bwr)
contour!(ax,x1range,x2range,funcplot; labels=true, levels,color=:black)
scatter!(ax, [-0.270845], [-0.923039]; color=:magenta, markersize=15)
scatter!(ax, [3.0, -2.805118, -3.779310, 3.584428], [2.0, 3.13132, -3.283136, -1.105196]; color=:black, markersize=15)

x1range_arrow=-6:1:6;
```

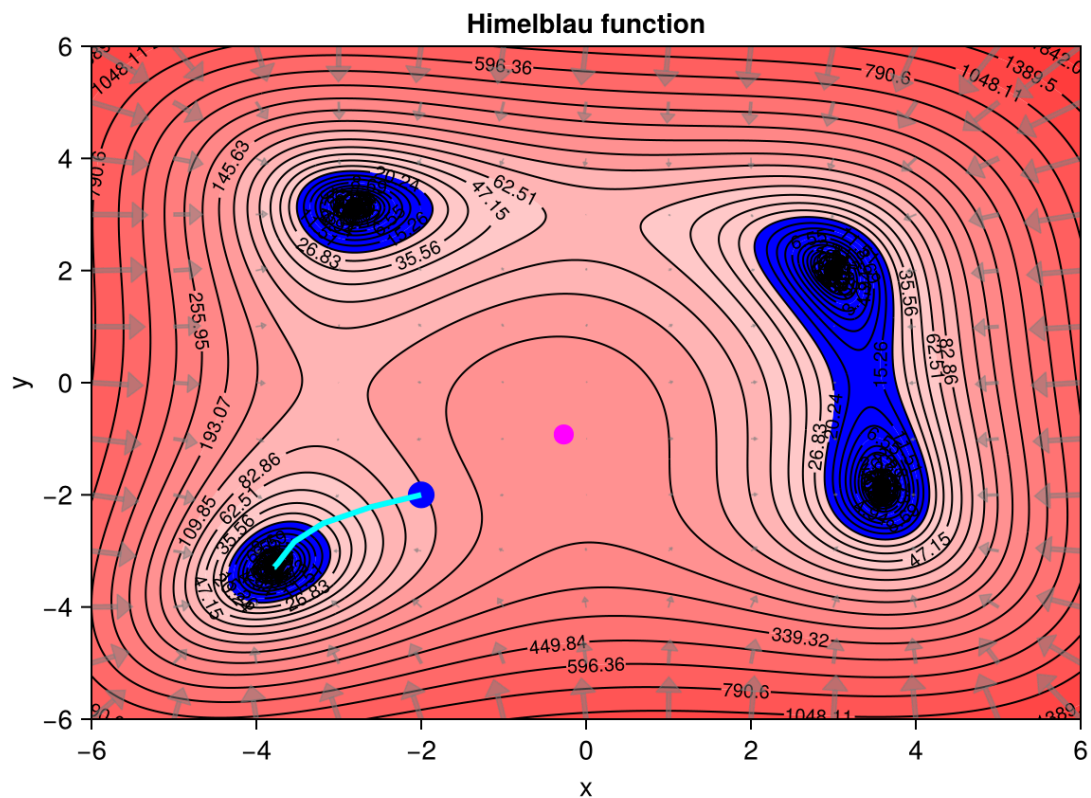
```
x2range_arrow=-6:1:6;
arrowplot = [Gradient_himmelblau([x1,x2]) for x1 in x1range_arrow, x2 in
arrows2d!(ax,x1range_arrow,x2range_arrow,-getindex.(arrowplot,1),-getinde
display(fig)
```



CairoMakie.Screen{IMAGE}

Example 2.4.3 with Standard Gradient Descent

```
In [11]: x0=[-2.0,-2.0] #initial guess of xm
alpha=0.01 #learning rate
data=[Point2D(x0...,himmelblau(x0))]
x=x0
for i in 1 : 20
    x-=alpha*Gradient_himmelblau(x)
    push!(data,Point2D(x...,himmelblau(x)))
end
scatter!(ax,x0[1],x0[2],color=:blue,markersize=20)
lines!(ax,getproperty.(data, :x1),getproperty.(data, :x2),color=:cyan,lin
fig
```



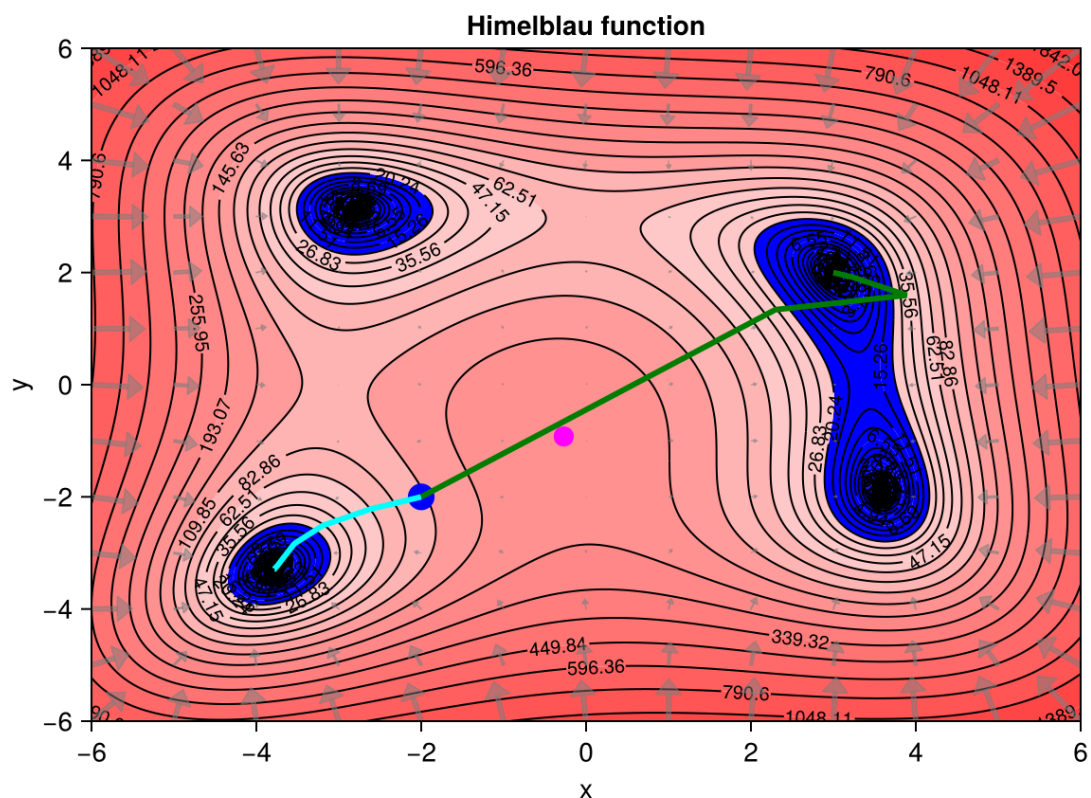
Now try Newton method with Himmelblau with initial guess $x_0 = [-2, -2]$

```
In [12]: x0=[-2.0,-2.0]#initial guess of x0

data_Newton=[Point2D(x0...,himmelblau(x0))]

x=x0
for i in 1 : 50
    x-=Hessian_himmelblau(x)\Gradient_himmelblau(x)
    push!(data_Newton,Point2D(x...,himmelblau(x)))
end

lines!(ax,getproperty.(data_Newton, :x1),getproperty.(data_Newton, :x2),l
fig
```



In [13]: data_Newton

```
51-element Vector{Point2D}:
 Point2D(-2.0, -2.0, 106.0)
 Point2D(2.295774647887324, 1.3380281690140845, 27.775159384379172)
 Point2D(3.890345134033711, 1.595352594656927, 33.1531457103013)
 Point2D(3.2392670282048375, 1.915129519506457, 1.9910600181057867)
 Point2D(3.0253020169071303, 1.9896539418099064, 0.020449388328767935)
 Point2D(3.000324004744422, 1.999889719422051, 3.376730016233299e-6)
 Point2D(3.00000005541271, 1.999999977956334, 9.744174283056258e-14)
 Point2D(3.0000000000000018, 1.9999999999999993, 1.1438483125704671e-28)
 Point2D(3.0, 2.0, 0.0)
 Point2D(3.0, 2.0, 0.0)
 ⋮
 Point2D(3.0, 2.0, 0.0)
 Point2D(3.0, 2.0, 0.0)
 Point2D(3.0, 2.0, 0.0)
 Point2D(3.0, 2.0, 0.0)
 Point2D(3.0, 2.0, 0.0)
 Point2D(3.0, 2.0, 0.0)
 Point2D(3.0, 2.0, 0.0)
 Point2D(3.0, 2.0, 0.0)
 Point2D(3.0, 2.0, 0.0)
```

Note that Newton method converges to a different location than the standard Gradient Descent method. This can happen, that you get different answers with different method!

Exercise Ex02: Run Newton method again but use the initial guess $(x_1, x_2) = (-0.5, -1.0)$. Can you explain what happens?

Exercise Ex03: Use the methods above to find the minimum of the Rosenbrock function. You are given that

$$f(x_1, x_2) = (1 - x_1)^2 + (x_2 - x_1^2)^2$$

$$\frac{\partial f}{\partial x_1}(x_1, x_2) = -2(1 - x_1) - 4x_1(x_2 - x_1^2)$$

$$\frac{\partial f}{\partial x_2}(x_1, x_2) = 2(x_2 - x_1^2)$$

$$\frac{\partial^2 f}{\partial x_1^2}(x_1, x_2) = 2 - 4x_2 + 12x_1^2$$

$$\frac{\partial^2 f}{\partial x_1 \partial x_2}(x_1, x_2) = -4x_1$$

$$\frac{\partial^2 f}{\partial x_2 \partial x_1}(x_1, x_2) = -4x_1$$

$$\frac{\partial^2 f}{\partial x_2^2}(x_1, x_2) = 2$$

The Adagrad and RMP prop methods

The momentum and Nesterov momentum uses a single value for the learning rate α for all components of $\vec{x}$. This might not be optimum for cases where you have some components of the gradients that are very large and some components that are very small (like the Rosenbrock function). Because the momentum and Nesterov momentum uses constant learning rate, the updates are very large for components of $\vec{x}$ with large gradients and small for components of $\vec{x}$ where the gradients that are small. The Adagrad and RMS prop method describe below adjusts the effective learning rate (so will have different learning rates for different components of $\vec{x}$) so that all components get updated at a similar magnitude.

The Adagrad (adaptive gradient) method uses different values of α for different components of x . When the partial derivative is large, it uses smaller values of α . The update equations for the Adagrad method can be written as

$$x_i^{k+1} = x_i^k - \frac{\alpha}{\epsilon + \sqrt{s_i^{(k)}}} g_i^{(k)}$$

where

$$s_i^{(k)} = \sum_{j=1}^k \left(g_i^{(j)} \right)^2$$

ϵ is a small number to prevent division by zero. Hence $s_i^{(k)}$ is the sum of the squared of the i 'th gradient from the first gradient up to k 'th step. One can think of the Adagrad method as the gradient descent method with effective learning rate of

$$\frac{\alpha}{\epsilon + \sqrt{s_i^{(k)}}}.$$

Note that the effective learning rate is monotonically decreasing since the definition of $s_i^{(k)}$ requires it to keep increasing at every iteration. This will eventually drive the effective learning rate to zero, which slows down the convergence of the Adagrad method.

Rosenbrock method with Adagrad

```
In [14]: rosenbrock(x)=(p[1] - x[1])^2 + p[2] * (x[2] - x[1]^2)^2

function Gradient_rosenbrock(x)
    d1=-2*(p[1] - x[1]) -4*x[1]*p[2] * (x[2] - x[1]^2)
    d2=2*p[2] * (x[2] - x[1]^2)
    return [d1,d2]
end

function Hessian_rosenbrock(x)
    return [2+12*p[2]*x[1]^2-4*x[2]*p[2] -4*x[1]*p[2];
            -4*x[1]*p[2] 2*p[2]]
end

x1range=-5:0.02:5;
x2range=-5:0.02:5;

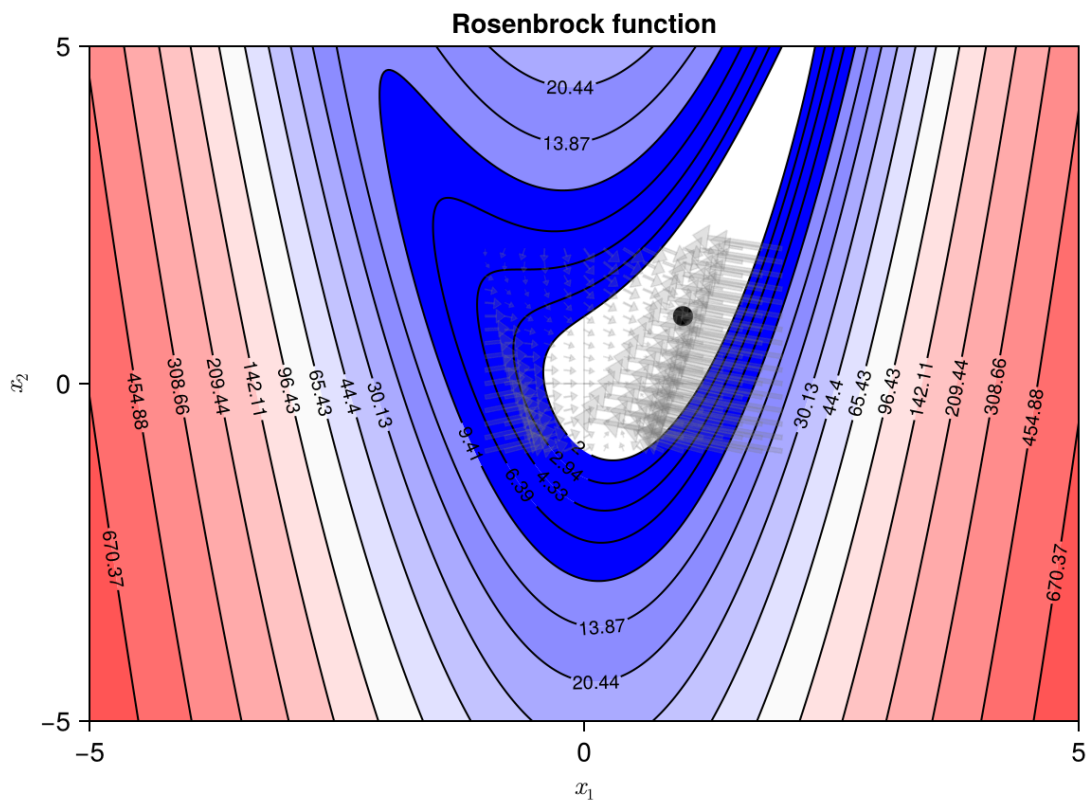
p=[1.0,1.0];

funcplot = [rosenbrock([x1,x2]) for x1 in x1range, x2 in x2range]

fig = Figure()
ax = Axis(fig[1, 1], xlabel = L"x_1", ylabel = L"x_2",title = "Rosenbrock
levels = 10.0.^range(0.3, 3.5; length=20)
contourf!(ax,x1range,x2range,funcplot; levels, colormap=:bwr)
contour!(ax,x1range,x2range,funcplot; labels=true, levels, color=:black)
scatter!(ax,[1],[1];color=:black,markersize=15)

x1range_arrow=-1:0.2:2;
x2range_arrow=-1:0.2:2;
arrowplot = [Gradient_rosenbrock([x1,x2]) for x1 in x1range_arrow, x2 in
arrows2d!(ax,x1range_arrow,x2range_arrow,-getindex.(arrowplot,1),-getinde

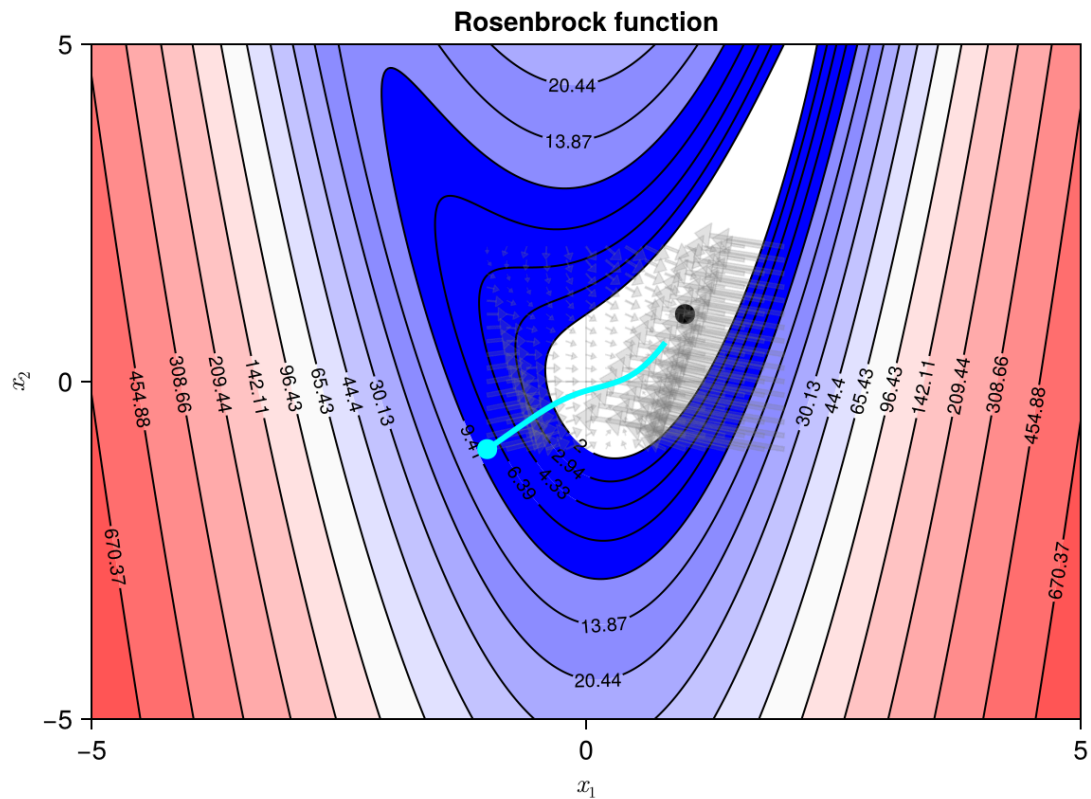
fig
```



```
In [15]: x0=[-1.0,-1.0] #initial guess of x0
alpha=0.1 #learning rate
s=[0.0,0.0]
epsilon=1.0e-8
data_ada=[Point2D(x0..., rosenbrock(x0))]

x=x0

for i in 1 : 500
    g=Gradient_rosenbrock(x)
    s .+= g.*g # sum of squared gradient
    x-=alpha*g ./ (sqrt.(s) .+ epsilon)
    push!(data_ada,Point2D(x..., rosenbrock(x)))
end
scatter!(ax,x0[1],x0[2];markersize=15,color=:cyan)
lines!(ax,getproperty.(data_ada, :x1),getproperty.(data_ada, :x2),linewidth=2)
fig
```



Example 2.4.4 with RMS prop method

The RMS prop method addresses the main drawback of the Adagrad method (the effective learning rate goes to zero!) by taking the weighted average of the previous two squared gradients. The update equation for the RMS prop method is

$$x_i^{k+1} = x_i^k - \frac{\alpha}{\epsilon + \sqrt{s_i^{(k+1)}}} g_i^{(k)}$$

where

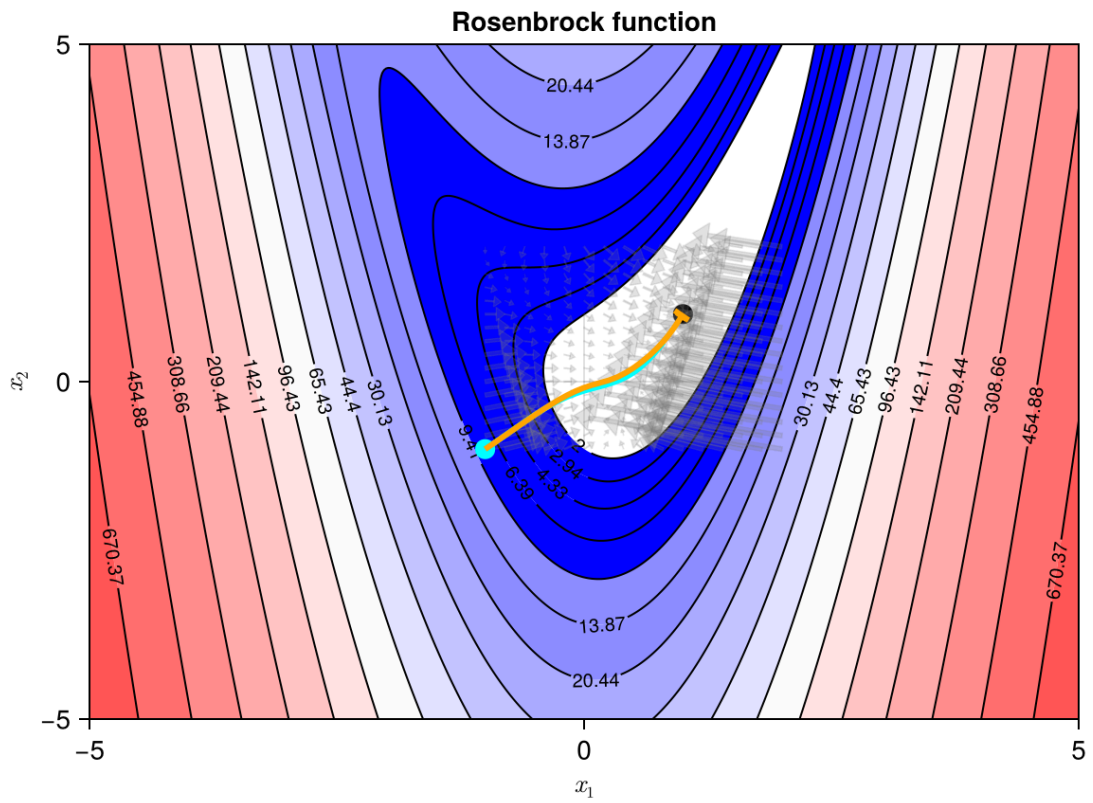
$$s_i^{(k+1)} = \gamma s_i^{(k)} + (1 - \gamma) g_i^{(k)} \times g_i^{(k)}$$

```
In [16]: x0=[-1.0,-1.0] #initial guess of xm
alpha=0.1 #learning rate
s=[0.0,0.0]
epsilon=1.0e-8
gamma=0.5 #decay
data_rms=[Point2D(x0..., rosenbrock(x0))]

x=x0

for i in 1 : 100
    g=Gradient_rosenbrock(x)
    s .= gamma*s + (1-gamma)*(g.*g)
    x-=alpha*g ./ (sqrt.(s) .+ epsilon)
    push!(data_rms,Point2D(x..., rosenbrock(x)))
end
```

```
lines!(ax, getproperty.(data_rms, :x1), getproperty.(data_rms, :x2), linewidth=2, color=:blue)
fig
```



In [17]:

In [18]: